Sentiment Analysis of Financial News

Riccardo Tellarini
Vasileios Lalos

1 Introduction

This project performs sentiment analysis on financial news headlines from the `Sentences_50Agree.txt` dataset, where each headline is labelled *positive*, *neutral*, or *negative*. The aim is to understand how dataset properties, preprocessing, and model choices shape model behaviour and errors. The main characteristics of the data that shape the analysis are a strong class imbalance and the short and domain-specific nature of the headlines.

Exploratory data analysis and selective preprocessing were performed first, word similarity was explored through a PMI-based method, and lastly the classification was performed using classical models (Naive Bayes and a feed-forward network), and a transformer model. A recurring theme is that the main difficulty is not separating positive from negative, but distinguishing genuine sentiment from neutral reporting.

2 Experimental Setup

All experiments used only the `Sentences_50Agree.txt` file, parsed by splitting each line at the last occurrence of `@`. Random procedures, including train/test splits and PMI target-word selection, used a fixed seed for reproducibility. Text processing relied on standard Python libraries such as `pandas`, `NLTK`, and `scikit-learn`. The preprocessing notebook produced a CSV containing the original text, the label, and three variants: `text_light`, `text_stopwords`, and `text_stemming`.

3 Exploratory Data Analysis

3.1 Dataset and Class Distribution

The dataset contains 4846 financial news headlines. Each headline is associated with one sentiment label. The class distribution is imbalanced, as shown in Table 1.

Table 1: Class distribution.

Class	Count	Percentage
Neutral	2879	59.4%
Positive	1363	28.1%
Negative	604	12.5%

The dominance of the neutral class is important for evaluation. A majority-class baseline that always predicts *neutral* would already achieve approximately 59% accuracy. Therefore, accuracy alone is not sufficient. Macro-averaged precision, recall, and F1-score are more informative in this setting because they compute an unweighted average over the classes, thus making performance on the minority *negative* class visible.

3.2 Text Length, Vocabulary and Sparsity

The headlines are short. Using the tokenization adopted in the exploratory notebook, the mean headline length is about 25.2 tokens and the median is 23 tokens. The maximum length is 111 tokens, but most examples are much shorter. Short texts provide limited context, which may make the distinction between neutral and sentiment-bearing headlines difficult, especially when sentiment depends on financial interpretation. For instance, the word *profit* is not necessarily positive: it becomes positive in expressions such as *profit rose*, but negative in expressions such as *profit fell* or *profit decreased*.

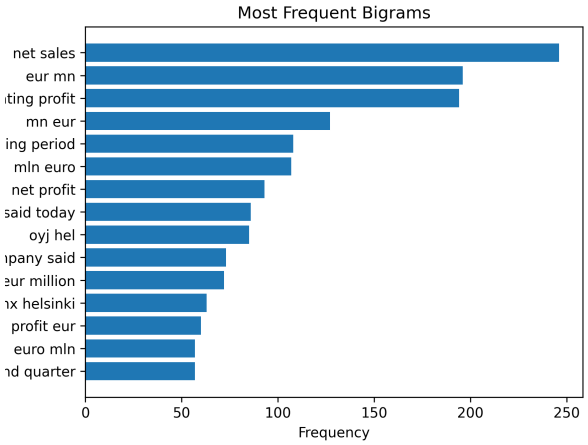

Figure 1: Most frequent bigrams after stop-word removal.

The corpus contains 122316 tokens and 10136 unique tokens in the exploratory tokenization. About 5113 words appear only once. This relatively large number of rare words suggests that bag-of-words and TF-IDF representations will be high-dimensional and sparse.

This sparsity is relevant for the later classification experiments, because classical models depend strongly on the quality of the extracted features. It is also relevant for PMI-based similarity, since rare co-occurrences can receive high PMI values and lead to unstable similarity results.

3.3 Frequent Words and Bigrams

The most frequent content words confirm the financial nature of the dataset. Frequent terms include *EUR*, *company*, *said*, *mn*, *sales*, *million*, *net*, *profit*, *operating*, and *market*.

The bigram analysis is as well domain-specific and informative. Frequent bigrams include *net sales*, *EUR mn*, *operating profit*, *net profit* and *net loss*. This may suggest that local word combinations are more informative than isolated words. For example, *profit* alone is ambiguous, while *profit rose* or *profit fell* provide clearer sentiment cues. This motivates n-gram features in classical models and explains why error analysis should consider local expressions rather than only individual words.

Table 2: Effect of preprocessing on token statistics.

Variant	Avg. tokens	Vocabulary	Tokens
`text_light`	21.24	10751	102939
`text_stopwords`	13.46	10503	65218
`text_stemming`	13.46	8285	65218

3.4 Numbers and Financial Symbols

Financial headlines often contain numbers, percentages, currencies and monetary quantities. In our dataset, 53.3% of headlines contain at least one number, 6.8% contain a percentage pattern, and 19.8% contain money-related terms or symbols. These domain-specific elements were treated as potentially informative rather than as noise.

4 Text Preprocessing

The preprocessing strategy was designed to be incremental: we defined three variants with increasing levels of normalization.

The three variants isolate the effect of increasing preprocessing strength. The `text_light` version is a conservative baseline preserving most lexical information, `text_stopwords` tests whether removing frequent non-informative words helps sparse representations, `text_stemming` tests whether a stronger vocabulary reduction helps.

The light preprocessing keeps words, numbers, percentages, and financial symbols whenever possible. This follows directly from the EDA: more than half of the headlines contain numbers.

The stop-word variant removes common English stop words but preserves words important for sentiment, such as *no*, *not*, *up*, *down*. This avoids an overly mechanical strategy, since negation and direction words can affect sentiment.

The stemming variant applies Porter stemming, reducing the vocabulary from 10503 to 8285 terms while keeping the same number of tokens (Table 2). It was included as an aggressive alternative, not as an obviously superior method: it can reduce sparsity but makes tokens less interpretable and may distort some financial terms (e.g. *company* becomes *compani*).

Some operations were intentionally not applied. Numbers, percentages, currencies, and financial units were preserved because they may express financial context. Lemmatization was not added as a separate variant, since stemming already provides an aggressive baseline. For transformer-based models, aggressive preprocessing is inappropriate because pretrained tokenizers are designed for raw text.

5 PMI-Based Word Similarity

5.1 Method

For PMI-based word similarity, a word-word co-occurrence matrix was constructed. Each row corresponds to a target word and each column to a context word. The context window size was set to one, so only the immediately preceding and following words were counted as context.

Table 3: PMI-based word similarity results.

Target word	Most similar words
ministry	infrastructure, sectors, republic, great, sawmill
working	produced, extended, independent, upm, restructuring
programs	black, ssh, operational, independent, model
needs	drive, gives, overview, comparable, holds
countries	regions, offices, players, northern, north
changed	magazines, places, fresh, closed, enso
introduce	version, step, clean, export, implementation
fifth	letter, took, consumption, transaction, lost
forest	process, paper, profile, heavy, construction
dec	nov, feb, oct, jan, alexandria

From the co-occurrence counts, we computed pointwise mutual information as:

$$PMI(w,c) = \log_2 \frac{P(w,c)}{P(w)P(c)} = \log_2 \frac{\text{count}(w,c) \cdot N}{\text{count}(w)\,\text{count}(c)},$$

where N is the total number of observed co-occurrences. Each word was then represented as a PMI vector over context words, and cosine similarity between PMI vectors retrieved the most similar words for ten randomly selected target terms.

Before constructing the PMI matrix, words occurring fewer than five times were removed since PMI is sensitive to rare words and the EDA revealed that rare words account for about half of the vocabulary.

5.2 Results and Interpretation

Table 3 reports the ten randomly selected target words and their most similar words.

The results should be interpreted as distributional similarity rather than direct synonymy. Since the context window is limited to the immediately adjacent words, PMI mainly captures local co-occurrence patterns. This may be suitable for financial headlines, as seen in the EDA. Unexpected similarities could be explained by the small corpus size, the narrow window, or rare co-occurrences.

6 Sentiment Classification

This section builds classical classifiers for the sentiment task and analyses their behaviour. We train a Multinomial Naive Bayes (NB) classifier and a feed-forward neural network (a multilayer perceptron, MLP), each with two feature representations, and we study the effect of class imbalance, of dropping the neutral class, and of the typical errors. The aim, following the EDA, is not to maximise accuracy but to understand *why* each modelling choice changes the results.

6.1 Setup

The dataset was split 80/20 with a stratified split (`random_state=42`) to preserve the 59/28/12 class proportions; without stratification the small negative class (12%) would make test metrics unstable. Features were fit on train and transformed on test to avoid leakage. We

Table 4: Naive Bayes with two representations and with a uniform prior.

Configuration	Acc.	Macro-P	Macro-R	Macro-F1	Neg. rec.
NB + BoW	0.72	0.69	0.64	0.66	0.53
NB + TF-IDF	0.68	0.73	0.45	0.45	0.06
NB + BoW (uniform prior)	0.68	0.63	0.65	0.64	0.63

report accuracy, macro-averaged precision, recall and F1, and confusion matrices; macro-F1 is the primary metric, since accuracy is dominated by the majority class. We fixed the `text_stopwords` preprocessing variant for the main grid: comparing it with `text_stemming` on NB + BoW gave macro-F1 0.66 vs. 0.65, so the more aggressive variant offered no gain while distorting financial terms.

6.2 Naive Bayes: Bag-of-Words vs. TF-IDF

We compare two representations for Naive Bayes: bag-of-words counts (BoW) and TF-IDF. The training BoW matrix has 8733 features and is about 99.9% sparse, consistent with the EDA. Table 4 reports the results.

Counter to the common assumption that TF-IDF is stronger, it *degraded* Naive Bayes sharply: macro-F1 fell from 0.66 to 0.45 and negative recall collapsed to 0.06. The mechanism is specific to NB, which scores a class as a product of a class prior and per-word likelihoods. TF-IDF down-weighting flattens the likelihood signal, so the high neutral prior (0.59) dominates and the model defaults to predicting neutral. Accuracy moved only slightly (0.72 to 0.68), while macro-F1 revealed that the model had effectively stopped predicting the minority classes — a direct illustration of the EDA warning.

6.3 Handling Class Imbalance

Since both representations collapsed toward neutral, we tested an explicit imbalance correction. NB has no `class_weight` parameter, but `fit_prior=False` replaces the data-estimated priors with a uniform prior. With BoW (Table 4, last row), negative recall rose from 0.53 to 0.63 and all three class F1-scores fell between 0.57 and 0.77, at the cost of a small accuracy drop (0.72 to 0.68). Macro-F1 itself barely changed (0.66 to 0.64), because the gain on the rare classes offset the loss on neutral. The value of this correction is therefore not a higher headline score but a model that no longer ignores the minority class.

6.4 Feed-forward Neural Network

We then trained an MLP (one hidden layer of 100 ReLU units, Adam, `max_iter=200`, `random_state=42`; ≈876,000 parameters) on the same representations. Table 5 places all four model×representation combinations side by side.

Two findings stand out. First, with BoW the neural network only *matched* Naive Bayes (0.67 vs. 0.66) despite its much larger capacity: for short headlines, word *presence* (*loss*, *profit*, *growth*) is nearly sufficient and word interactions add little. Second, TF-IDF devastated NB (0.45) but left the neural network almost unaffected (0.65; negative recall 0.54 vs. NB's 0.06), because the network has no class prior to be overwhelmed — it learns its weights directly by gradient descent. Representation effects are thus *not absolute*: they interact with the classifier.

Table 5: Macro-F1 for the full classifier × representation grid.

	BoW	TF-IDF
Naive Bayes	0.66	0.45
Neural network	0.67	0.65

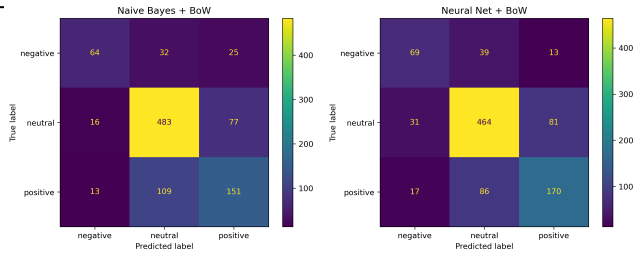

Figure 2: Confusion matrices (multiclass, BoW): Naive Bayes (left) and neural network (right). Both confuse positives and negatives with the neutral majority rather than with each other.

6.5 Error Analysis

We examined the misclassified test examples of the best multiclass model (NN + BoW, 267 errors out of 970). The confusion matrices (Figure 2) show that errors are overwhelmingly confusions with the neutral class rather than polarity flips: positives and negatives are rarely confused with each other. The difficulty is separating weak sentiment from neutral reporting.

The most frequent error type — true positives predicted as neutral — reveals a consistent pattern: these headlines contain almost no overtly positive *words*. We group the failures into four categories:

- **Sentiment in events/numbers:** e.g. "*awarded a significant order for 292 cranes*". The signal is the financial event, not a sentiment word.
- **Implicit/connotative:** e.g. "*moving to new headquarters signifies the beginning of a new era*". Positive by implication, with no positive keywords.
- **Negation / polarity reversal:** e.g. "*Stora Enso… was acquitted of charges of… price-fixing*". The sentence is full of negative words; *acquitted* reverses them, but bag-of-words ignores word order.
- **Domain knowledge required:** e.g. being listed among "*sustainable / pioneer companies*" requires world knowledge to read as positive.

These categories follow directly from the data properties — short, domain-specific headlines with implicit sentiment — and from the bag-of-words assumption that discards word order. Each failure mode (word order, world knowledge, context) is precisely what a pretrained contextual model is designed to capture, motivating Section 7.

Table 6: Macro-F1 of all models in both settings; for DistilBERT, multiclass accuracy is 0.82 and multiclass negative recall is 0.86.

Model	Multiclass	Binary
NB + BoW	0.66	0.81
NN + BoW	0.67	0.83
DistilBERT	0.81	0.95

6.6 Binary vs. Multiclass

We repeated the task in a binary setting by removing the neutral class (1967 headlines, positive 69% / negative 31%), which also reduces the imbalance. Both classical models gain about +0.15 macro-F1 (NB 0.66 to 0.81, NN 0.67 to 0.83; see the classical rows of Table 6), and negative recall recovers strongly (NB 0.53 to 0.70, NN 0.57 to 0.74). The interpretation matters more than the numbers: every error category above was a confusion *with neutral*, so deleting the neutral class removes precisely the dominant failure mode. The ∼0.15 gain quantifies how much of the multiclass difficulty was due to the neutral class alone — the binary setting removes the hard part. The neural network again only marginally beats Naive Bayes, confirming that for this dataset a simple, interpretable, instantly-trained NB is a strong baseline.

7 Pre-trained Language Models

The classical error analysis identified failure modes — negation, implicit sentiment, financial events, world knowledge — that bag-of-words cannot capture because it discards word order and has no prior linguistic knowledge. We now fine-tune a pretrained transformer designed for exactly these limitations.

7.1 Model and Training Setup

We fine-tuned `distilbert-base-uncased` (≈67M parameters) using its own subword tokenizer. Unlike the classical models, the transformer was applied to the *raw* headlines: aggressive cleaning would discard information the pretrained tokenizer expects, since the model was pretrained on natural English. We used the same 80/20 stratified split and seed (42) as in Section 6 for direct comparability. Sequences were padded/truncated to 128 tokens (the EDA maximum is ≈100). Training used 3 epochs, batch size 16, and a learning rate of 2×10^{-5} — small enough to adapt the model to the financial domain while preserving the knowledge from pretraining. Fine-tuning ran on a free Google Colab T4 GPU.

7.2 Results and Comparison

Table 6 compares the fine-tuned transformer with the best classical models in both settings. The classical rows are repeated from Section 6 for direct comparison.

In the multiclass setting the transformer raised macro-F1 from 0.67 to 0.81 (+0.14). The improvement is concentrated where the classical models failed: negative recall rose from 0.53 to 0.86, positive recall from 0.55 to 0.79, and the dominant classical error (true positives predicted as neutral) dropped from 109 cases for NB to 50. The transformer therefore does not just score higher; it repairs the specific failure modes documented in the error analysis. Reading word order lets it handle negation such as *acquitted*; pretrained world knowledge lets it recognise that a won contract or a capital increase is positive; and contextual embeddings capture implicit, connotative sentiment such as "a new era". This confirms that contextual, pretrained representations capture sentiment that bag-of-words counting cannot.

In the binary setting the transformer reached macro-F1 0.95, with only 18 errors out of 394 test headlines and negative recall 0.95. Validation macro-F1 stabilised after one to two epochs and dipped slightly at the third, indicating three epochs is sufficient and further training would risk overfitting. The two effects compound: removing the neutral class and using a contextual model each help, and together they yield near-ceiling performance.

7.3 Computational Cost

These gains come at a cost. Naive Bayes trains in under a second on CPU and is fully interpretable through its per-word likelihoods; the MLP trains in seconds. The transformer required a GPU — three epochs took ≈1 minute on a Colab T4 but would take far longer on CPU — and is essentially a black box. For deployments that value transparency, speed, or limited hardware, the classical models remain attractive; the transformer is justified when maximising performance on difficult, context-dependent cases is the priority.

8 Conclusion

This project showed that methodological choices matter more than raw scores. The EDA revealed a strongly imbalanced, short, domain-specific dataset, which made macro-F1 and confusion matrices essential and motivated domain-aware preprocessing. The classification experiments demonstrated that representation effects interact with the classifier: TF-IDF collapsed Naive Bayes but not the neural network, and a high-capacity network barely outperformed simple word counting, indicating that word presence is nearly sufficient for these headlines. The error analysis traced the remaining failures to negation, implicit sentiment, and required domain knowledge. Fine-tuning DistilBERT addressed exactly these failures, improving macro-F1 from 0.67 to 0.81 (multiclass) and reaching 0.95 in the binary setting, at the cost of higher computation and lower interpretability. The clearest single lesson is that the central difficulty of the task was not distinguishing positive from negative, but separating genuine sentiment from neutral financial reporting.